





# Faculty of Engineering

# Electronics and Communications Department

# Communications Project 3

| Name | section | ID |
|---|---|---|
| سهيله احمد محمد عبدالمجيد | 2 | 9220377 |
| رؤي عاطف محمد | 2 | 9220291 |
| منار كمال مرسي | 4 | 9220869 |

**Role of Members**

| | |
|---|---|
| **رؤي عاطف محمد** | Data generation + Mapper implementation ($1^{st}$ 4 tasks) |
| **سهيله احمد محمد** | Demapper implementation ($1^{st}$ 4 tasks) + BFSK Task |
| **منار كمال مرسي** | Noise Addition + BER calculations ($1^{st}$ 4 tasks) |

# Introduction

This project involves simulating a single-carrier communication system, consisting of a mapper, AWGN channel, demapper, and BER calculator. It supports various modulation schemes like BPSK, QPSK, and 16QAM, and uses MATLAB to simulate noise and evaluate bit error rates.

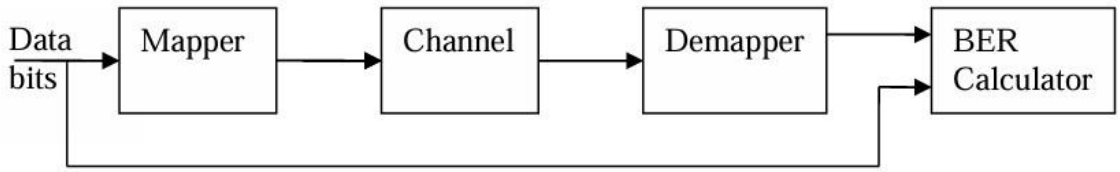



*Figure 1: single carrier communication system*

# Mapper

- In the mapper stage, the data bits are turned into symbols according to the modulation type (1 bit per symbol for BPSK,2 bits per symbol for QPSK, 3 bits per symbol for 8PSK, and 4 bits per symbol for QAM) .
- Symbols are converted to gray encoding form to minimize the BER.

# Single constellation

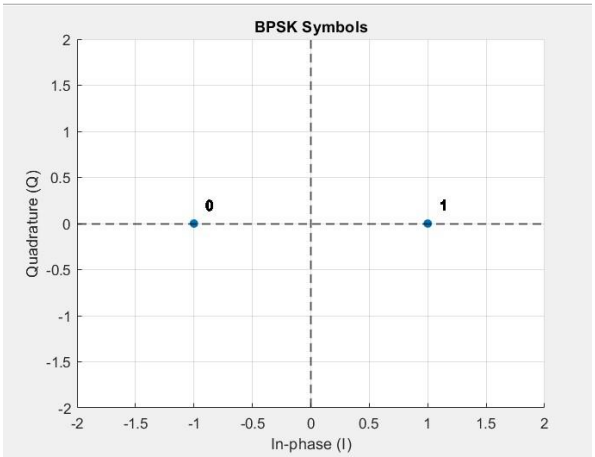



*Figure 2: BPSK Constellation*

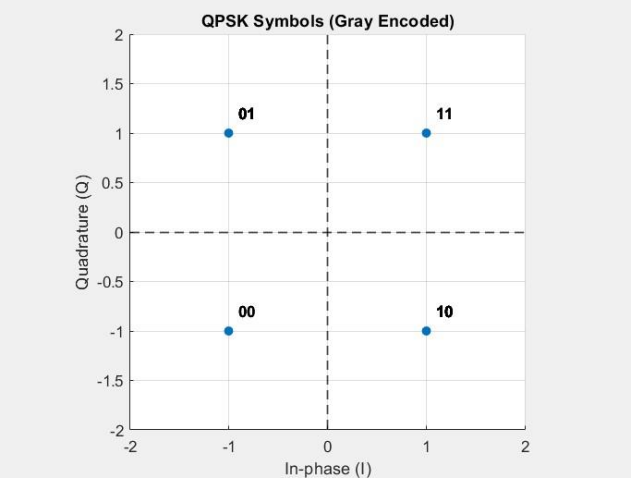



*Figure 3: QPSK Gray Constellation*

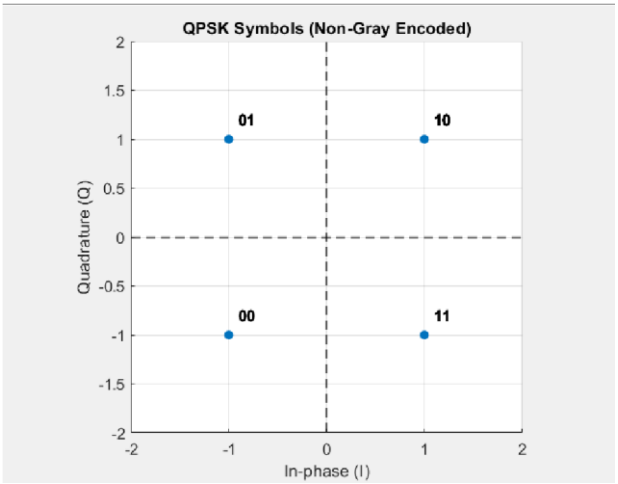



*Figure 4: QPSK Non-Gray Constellation*

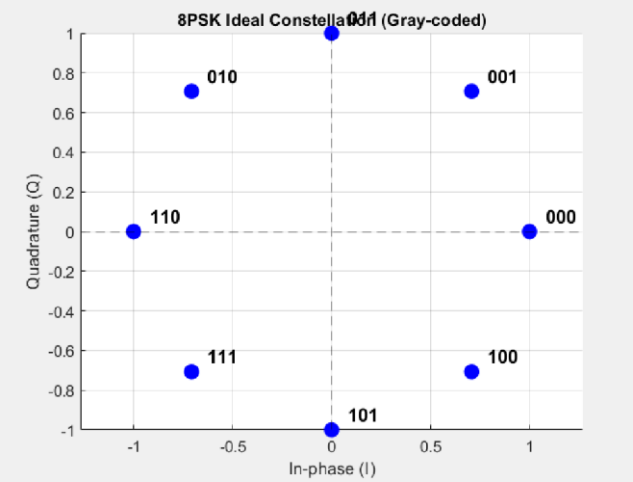



*Figure 5: 8-PSK Constellation*

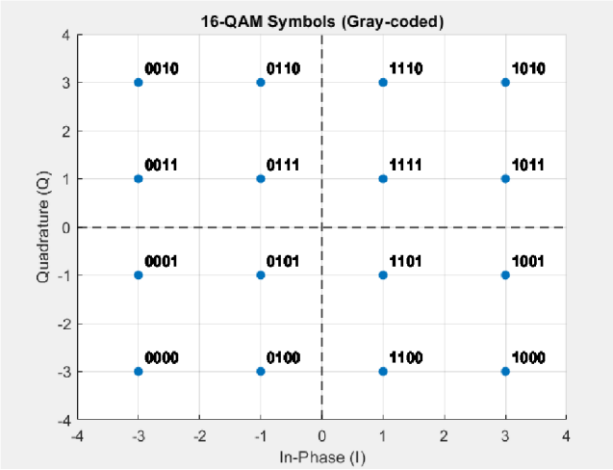
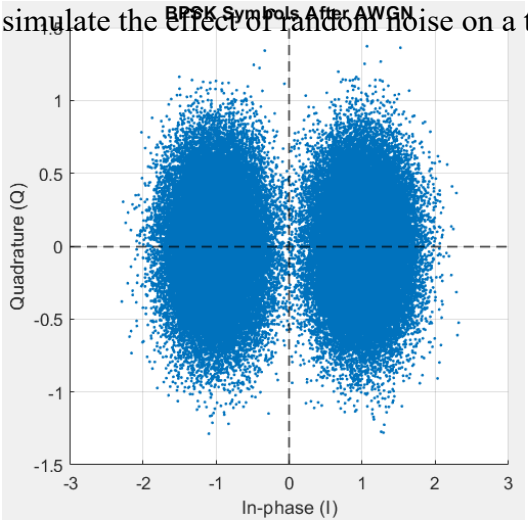
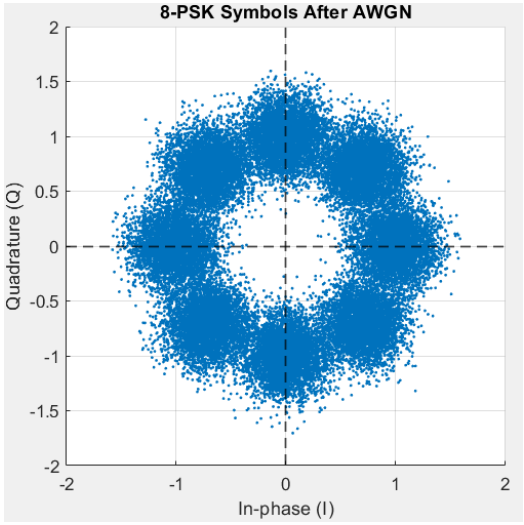



*Figure 10: 16-QAM Constellation*

## Channel and Noise

The AWGN (Additive White Gaussian Noise) channel is used in communication systems to simulate the effect of random noise on a transmitted signal.



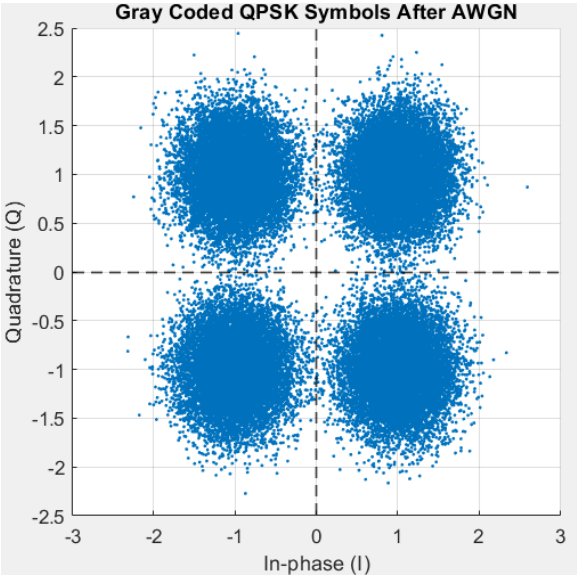
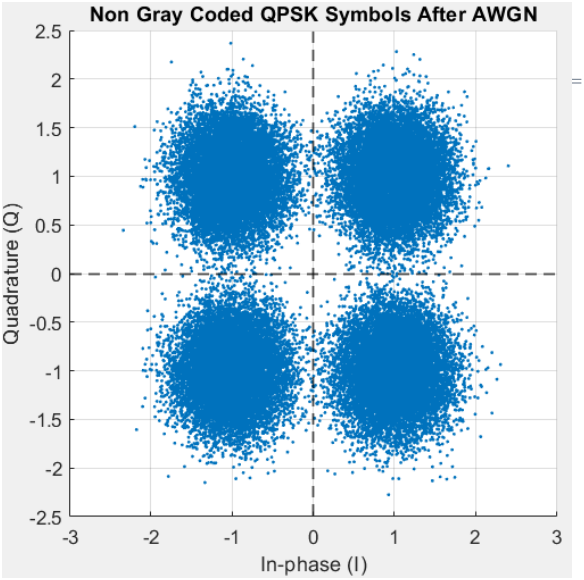



*Figure 9: Gray coded QPSK after AWGN at SNR = 5*




*= 5*

*Figure 10: Non-Gray coded QPSK after AWGN at SNR = 5*

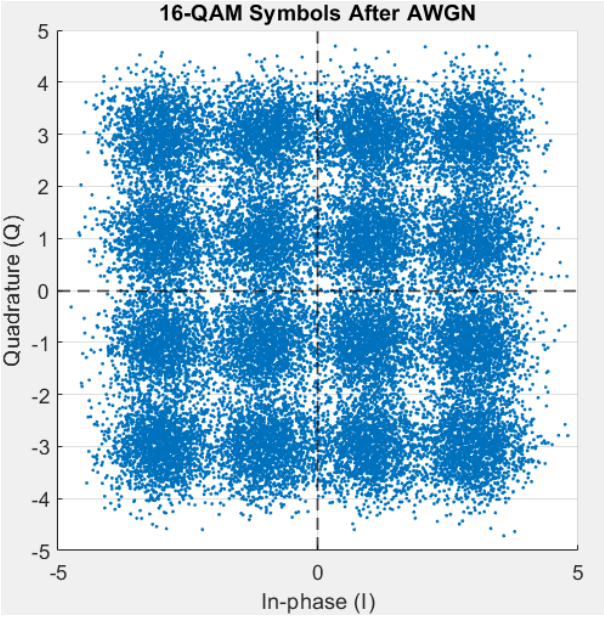



*Figure 19: 16-QAM symbols after AWGN at SNR = 5*

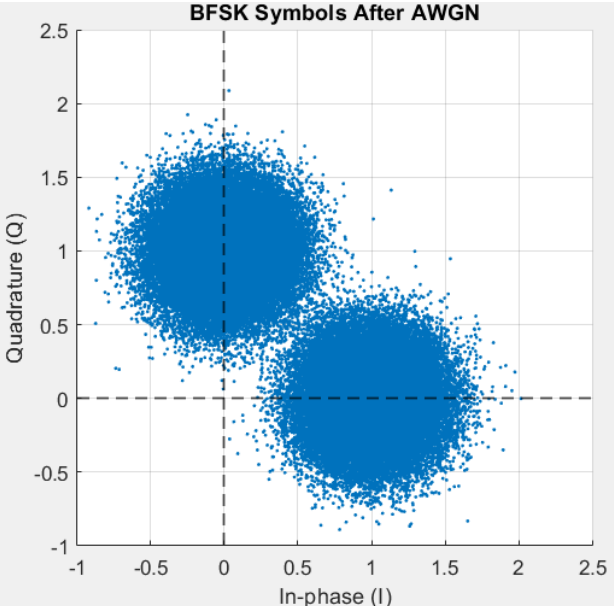



*Figure 15: BFSK symbols after AWGN at SNR = 10*

## Demapper

The demapper estimates transmitted symbols from received ones using the modulation scheme, converts them back into bits, and compares them with the original bit stream to calculate the Bit Error Rate (BER).

# BER

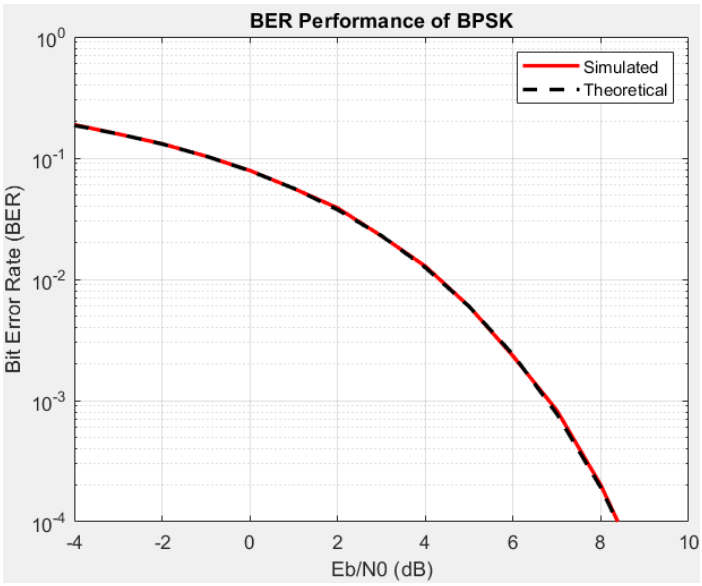



*Figure 3: BER of BPSK*

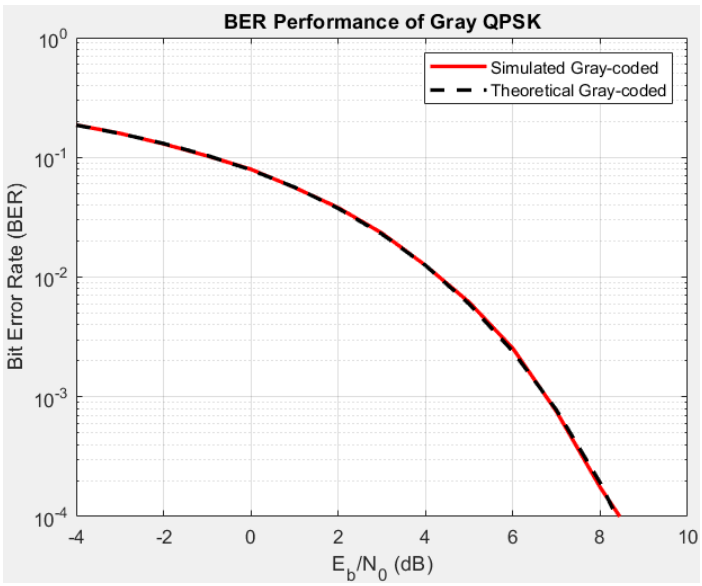



*Figure 14: BER of Gray coded QPSK*

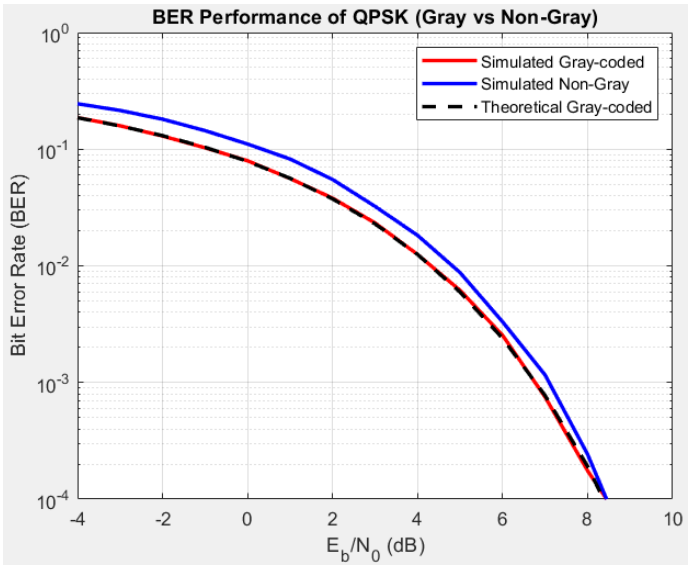



*Figure 15: BER of Non-Gray coded QPSK*

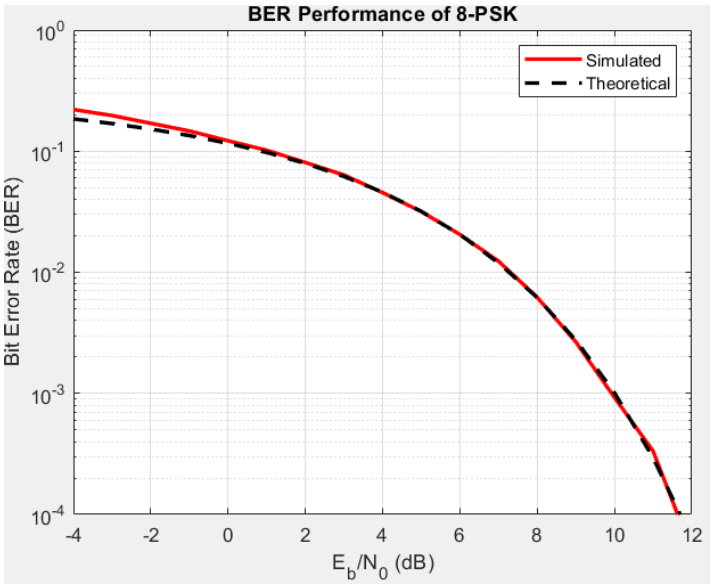



*Figure 16: BER of 8-PSK*

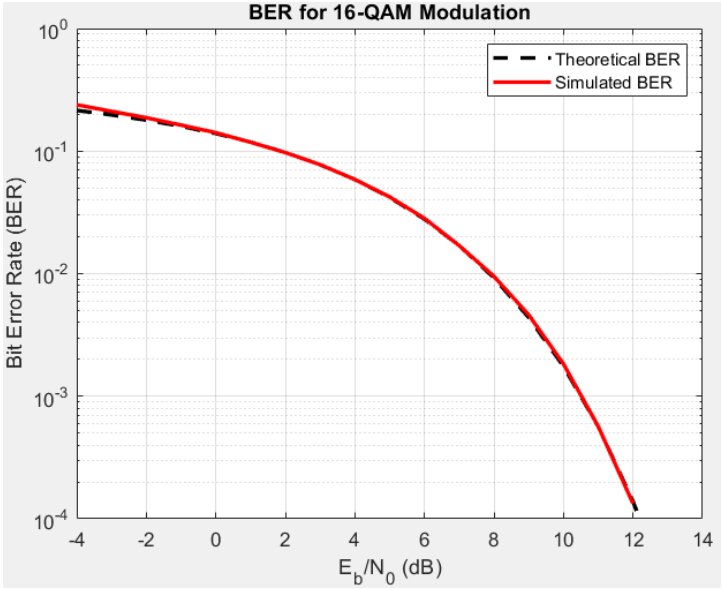



*Figure 33: BER of 16-QAM*

# Results and Analysis

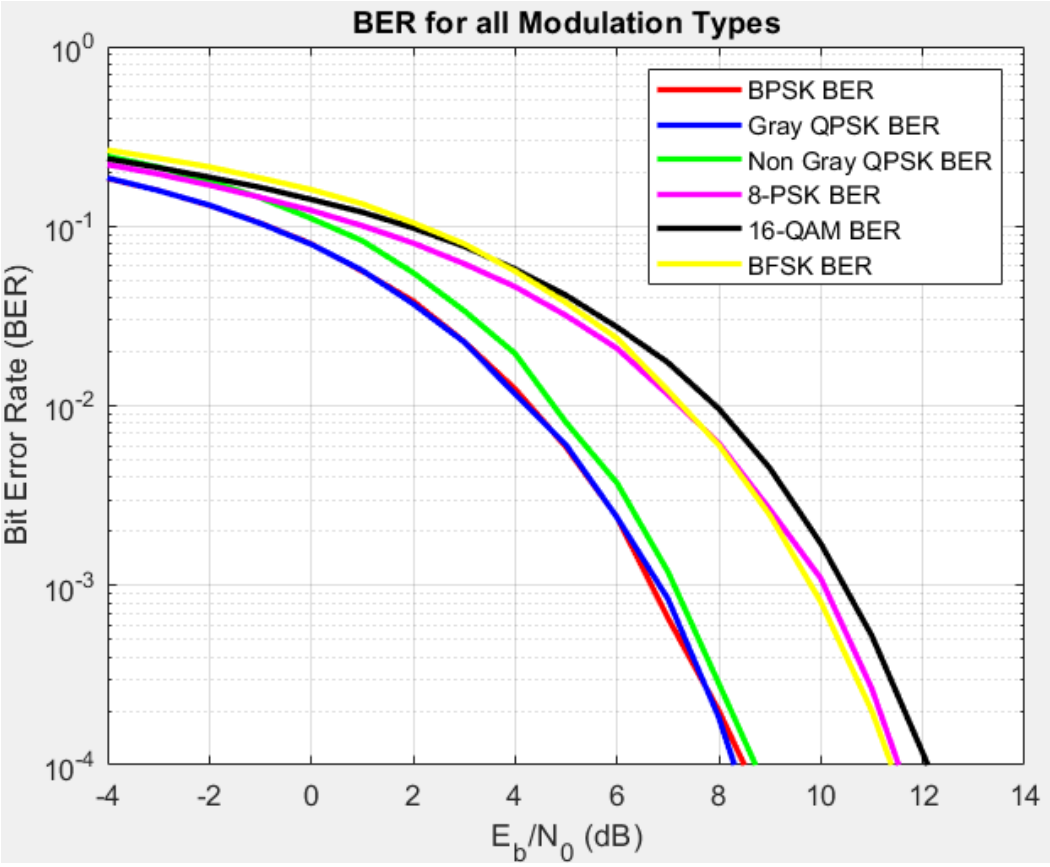



*Figure 41: BER for all modulation types*

Comments on results

From the plot, we can see that BPSK and Gray-coded QPSK achieve the lowest BER among the compared modulation schemes, making them preferable when minimizing bit error rate.

| *Modulation type* | *Theoretical BER* |
|---|---|
| ***BPSK*** | $\frac{1}{2}erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$ |
| ***QPSK*** | $\frac{1}{2}erfc\left(\sqrt{\frac{E_b}{N_0}}\right)$ |
| ***8-PSK*** | $\frac{1}{3}erfc\left(\sqrt{\frac{3E_b}{N_0}}sin\left(\frac{\pi}{8}\right)\right)$ |
| ***16-QAM*** | $\frac{3}{8}erfc\left(\sqrt{\frac{E_b}{2.5N_0}}\right)$ |
| ***BFSK*** | $\frac{1}{2}erfc\left(\sqrt{\frac{E_b}{2N_0}}\right)$ |

## BFSK Part

Consider the BFSK signal given by

$$s_i(t) = \begin{cases} \sqrt{\frac{2E_b}{T_b}}\cos\left(2\pi f_i t\right), & 0 \le t \le T_b \\ 0 & \text{otherwise} \end{cases}$$

The Tx frequency is $f_i=(n_c+i)/T_b \quad , i=1,2$

## 1) What are the basis functions of the signal set?

$$\Phi_i = \sqrt{\frac{2}{T_b}} \cos(2\pi f_i t)$$

$$\therefore \Phi_1 = \sqrt{\frac{2}{T_b}} \cos(2\pi f_1 t) \qquad \Phi_2 = \sqrt{\frac{2}{T_b}} \cos(2\pi f_2 t)$$

We can see that the basis functions look like the transmitted signals but they are normalized by the energy.

## 2) Write an expression for the baseband equivalent signals for this set, indicating the carrier frequency used.

$$S_1(t) = \sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t)$$

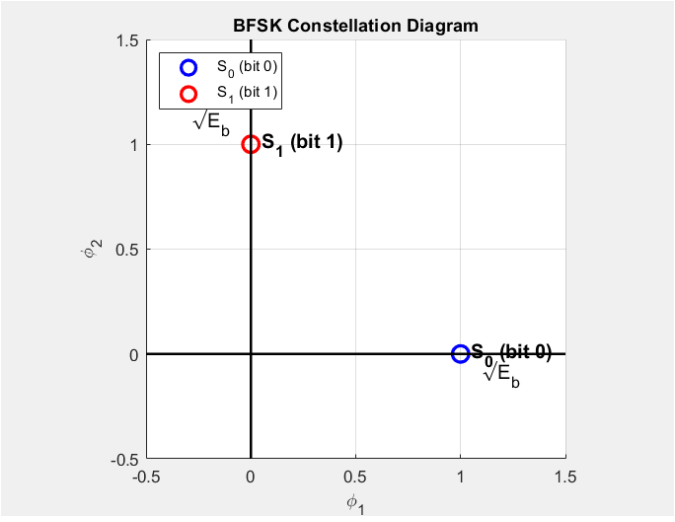
$$S_2(t) = \sqrt{\frac{2E_b}{T_b}} \cos(2\pi(f_c+\Delta f)t) = \sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t)\cos(2\pi\Delta f t) - \sqrt{\frac{2E_b}{T_b}} \sin(2\pi f_c t)\sin(2\pi\Delta f t)$$

$$\therefore S_{1BB}(t) = \sqrt{\frac{2E_b}{T_b}}$$

$$S_{1BB}(t) = \sqrt{\frac{2E_b}{T_b}} [\cos(2\pi\Delta f t) + j\sin(2\pi\Delta f t)]$$

We can assume here that the carrier frequency, $f_c = f_1$

## Plots:



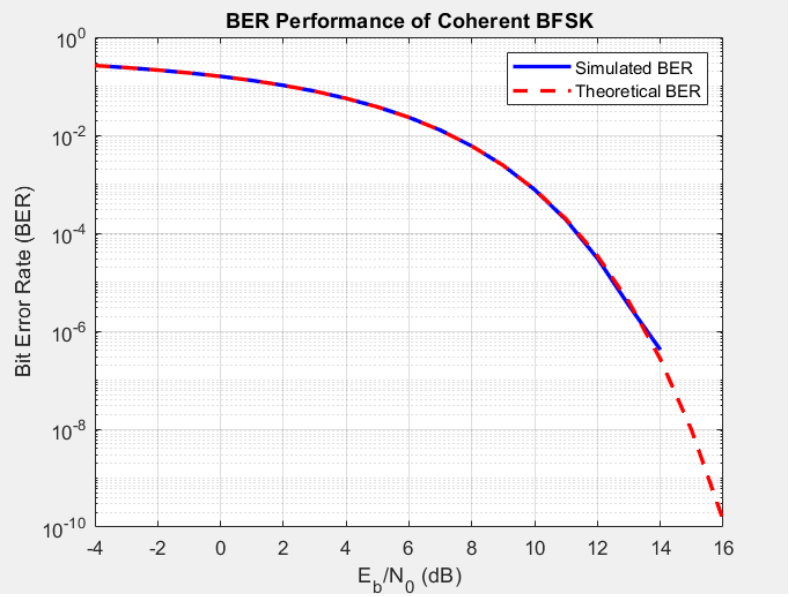



*Figure 45: BFSK Constellation*

*Figure 46: BER of BFSK*

## Comments:

We can see that the theoretical and simulated BER align perfectly.

## PSD plot for BFSK:

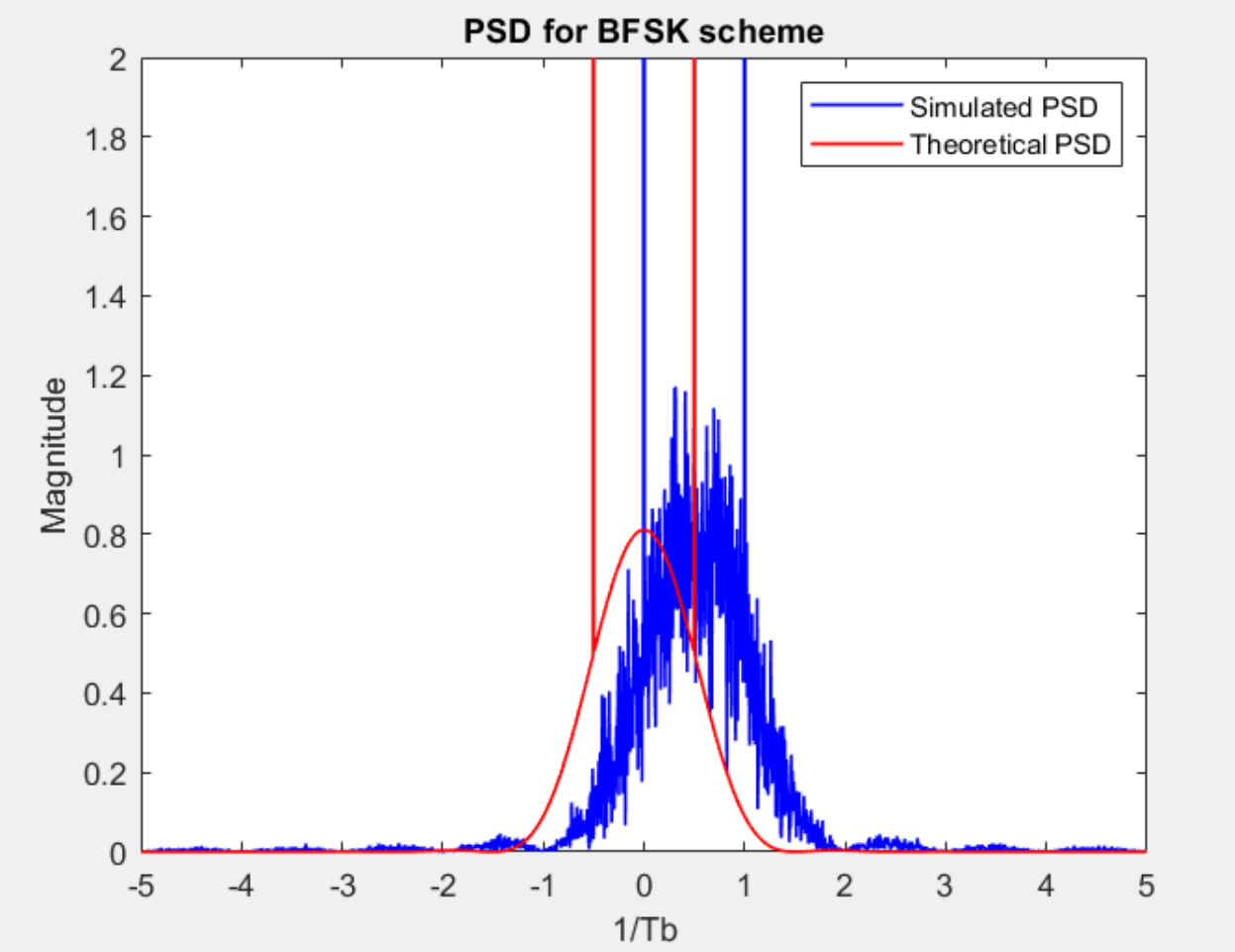



*Figure 50: PSD for BFSK*

## Comment:

The figure shows the Power Spectral Density (PSD) for the BFSK modulation scheme, comparing the simulated PSD (in blue) with the theoretical PSD (in red). As expected, the simulated PSD closely follows the general shape of the theoretical PSD.

We generated an ensemble with 200 bits and 4000 realizations to calculate the simulated PSD and we used this formula to get the theoretical PSD:

$S_{BB}(f) = \frac{2E_b}{T_b}\left[\delta\left(f - \frac{1}{2T_b}\right) + \delta\left(f + \frac{1}{2T_b}\right)\right] + \frac{8E_b cos^2(\pi T_b f)}{\pi^2(4T_b{}^2 f^2 - 1)}$ (This formula was mentioned in the tutorial).

## The code

```
%%
.....................................................Paramet
ers.....................................%%
N = 120000; % Number of bits
Eb = 1; % Energy per bit
EbNo_dB = -4:1:14; % Eb/No range in dB
M_BPSK = 2;  % BPSK modulation order
M_QPSK = 4;  % QPSK modulation order
M_8PSK = 8;  % 8PSK modulation order
M_QAM = 16;  % 16-QAM modulation order
%% ..............................................Generate
binary data.....................................%%
data_bits = randi([0 1], N, 1);
%%
.................................................BPSK......
.............................................%%

%.................................................Mapper..
.............................................%
%  Group bits for BPSK (1 bit per symbol)
bpsk_bits = groupBitsForMapping(data_bits, 2);  % 1 bit per
row
%  Apply Gray encoding
% For 1-bit input, Gray code is the same as the input
gray_bits = xor(bpsk_bits, floor(bpsk_bits/2));  % works
generally
%  Convert Gray code to decimal (trivial for 1 bit)
gray_decimal = gray_bits;  % For BPSK, it's 0 or 1
%  Map to BPSK symbols: 0 ? +1, 1 ? -1
bpsk_symbols =  2 * gray_decimal - 1;
% convert the bits to I+QJ form
bpsk_complex_symbols = bpsk_symbols + 0j;
% Preallocate BER array
BER_bpsk = zeros(1, length(EbNo_dB));
for k = 1:length(EbNo_dB)
    %..................................... Add noise at this
Eb/No.....................................%
```

```
    rx_bpsk = AWGN_Channel(bpsk_complex_symbols,
EbNo_dB(k), Eb, M_BPSK);
    %
.....Demapper.............
.....%
    BPSK_demap = zeros(length(data_bits),1);
    for i = 1:length(rx_bpsk)
        if real(rx_bpsk(i)) > 0
            BPSK_demap(i) = 1;
        else
            BPSK_demap(i) = 0;
        end
    end
    % .............................BER
calculation.....................................
.....%
    BER_bpsk(k) = BER_Calculation(BPSK_demap, data_bits,
N);
end
    % ..............................theoretical
BER.....................................................%
theoretical_BER_bpsk = 0.5 * erfc( sqrt(10.^(EbNo_dB/10))
);
% plot BER
figure;
semilogy(EbNo_dB, BER_bpsk, 'r-', 'LineWidth', 2); hold on;
semilogy(EbNo_dB, theoretical_BER_bpsk, 'k--', 'LineWidth',
2);
grid on;
xlabel('Eb/N0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER Performance of BPSK');
legend('Simulated', 'Theoretical');
ylim([1e-4 1]);

%% ..................................................
QPSK.....................................................%%

%.....................................................Mapper.
.....................................%
% Group bits for BPSK (1 bit per symbol)
qpsk = groupBitsForMapping(data_bits, M_QPSK);
% Apply gray encoding
gray_bits = xor(qpsk, floor(qpsk/2));
```

```
% Convert each 2-bit row to decimal (0-3)
gray_decimal = gray_bits(:,1)*2 + gray_bits(:,2);
% Preallocate complex symbol array
qpsk_symbols = zeros(length(gray_decimal), 1);
for k = 1:length(gray_decimal)
    switch gray_decimal(k)
        case 0
            qpsk_symbols(k) = -1 + -1j;      % 00
        case 1
            qpsk_symbols(k) = -1 + 1j;     % 01
        case 2
            qpsk_symbols(k) = 1 - 1j;     % 11
        case 3
            qpsk_symbols(k) = 1 + 1j;      % 10
    end
end
% Preallocate BER array
BER_qpsk_gray = zeros(1, length(EbNo_dB));
for k = 1:length(EbNo_dB)
    % .................................Gray encoded QPSK
noise...........%
    rx_qpsk_gray = AWGN_Channel(qpsk_symbols, EbNo_dB(k),
Eb, M_QPSK);
    %.....................QPSK (Gray)
Demapper.........................%
    QPSK_Gray_demap = zeros(2*length(rx_qpsk_gray), 1);  %
2 bits per symbol
    for i = 1:length(rx_qpsk_gray)
        real_part = real(rx_qpsk_gray(i));
        imag_part = imag(rx_qpsk_gray(i));

        if (real_part > 0) && (imag_part > 0)        % (+,+)
            QPSK_Gray_demap(i) = 3;
        elseif (real_part > 0) && (imag_part < 0)     % (+,-
)
            QPSK_Gray_demap(i) = 2;
        elseif (real_part < 0) && (imag_part > 0)     % (-
,+)
            QPSK_Gray_demap(i) = 1;
        elseif (real_part < 0) && (imag_part < 0)     % (-,-
)
            QPSK_Gray_demap(i) = 0;
        end
    end
```

```
    QPSK_Gray_demap = de2bi(QPSK_Gray_demap, 2, 'left-
msb');
    QPSK_Gray_demap = QPSK_Gray_demap.'; % Transpose
    QPSK_Gray_demap = QPSK_Gray_demap(:); % Reshape into
column

    %......................... Calculate
BER.........................%
    BER_qpsk_gray(k) = BER_Calculation(QPSK_Gray_demap,
data_bits, N);

end

%............................ theoretical
BER.............................%
theoretical_BER_qpsk = 0.5 * erfc( sqrt(10.^(EbNo_dB/10))
);
% plot the BER
figure;
semilogy(EbNo_dB, BER_qpsk_gray, 'r-', 'LineWidth', 2);
hold on;
semilogy(EbNo_dB, theoretical_BER_qpsk, 'k--', 'LineWidth',
2);
grid on;
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER Performance of Gray QPSK');
legend('Simulated Gray-coded', 'Theoretical Gray-coded');
ylim([1e-4 1]);

% Non-gray encoded symbols (direct from bits, no xor)
binary_decimal = qpsk(:,1)*2 + qpsk(:,2);  % Regular binary
to decimal

% Map using regular binary values (00, 01, 10, 11)
qpsk_symbols_nongray = zeros(length(binary_decimal), 1);
for k = 1:length(binary_decimal)
    switch binary_decimal(k)
        case 0
            qpsk_symbols_nongray(k) = -1 - 1j;  % 00
        case 1
            qpsk_symbols_nongray(k) = -1 + 1j;  % 01
        case 2
```

```
            qpsk_symbols_nongray(k) = 1 +1j;    % 10
        case 3
            qpsk_symbols_nongray(k) = 1 - 1j;    % 11
    end
end
% Preallocate BER array
BER_qpsk_nongray = zeros(1, length(EbNo_dB));
for i = 1:length(EbNo_dB)
    %.............. Non-Gray encoded QPSK noise
.......................%
    rx_qpsk_non_gray = AWGN_Channel(qpsk_symbols_nongray,
EbNo_dB(i), Eb, M_QPSK);

   %.....................QPSK (normal)
Demapper.........................%
    QPSK_normal_demap  =
zeros(length(qpsk_symbols_nongray), 1);  % 1 column complex
values

    for k = 1:length(qpsk_symbols_nongray)
        if(real(rx_qpsk_non_gray(k)) > 0 &&
imag(rx_qpsk_non_gray(k)) < 0)
            QPSK_normal_demap(k) = 3;
        elseif(real(rx_qpsk_non_gray(k)) > 0 &&
imag(rx_qpsk_non_gray(k)) > 0)
            QPSK_normal_demap(k) = 2;
        elseif(real(rx_qpsk_non_gray(k)) < 0 &&
imag(rx_qpsk_non_gray(k)) > 0)
            QPSK_normal_demap(k) = 1;
        elseif(real(rx_qpsk_non_gray(k)) < 0 &&
imag(rx_qpsk_non_gray(k)) < 0)
            QPSK_normal_demap(k) = 0;
        end
    end
    % Convert the normal binary code index to 2-bit binary
    QPSK_normal_demap = de2bi(QPSK_normal_demap, 2, 'left-
msb');
    QPSK_normal_demap = QPSK_normal_demap.'; % transpose
    QPSK_normal_demap = QPSK_normal_demap(:); % reshape
into column
    % Calculate BER
    BER_qpsk_nongray(i) =
BER_Calculation(QPSK_normal_demap, data_bits, N);
end
```

```
%.......................... theoretical
BER.............................%
theoretical_BER_qpsk = 0.5 * erfc( sqrt(10.^(EbNo_dB/10))
);
% plot the BER
figure;
semilogy(EbNo_dB, BER_qpsk_gray, 'r-', 'LineWidth', 2);
hold on;
semilogy(EbNo_dB, BER_qpsk_nongray, 'b-', 'LineWidth', 2);
semilogy(EbNo_dB, theoretical_BER_qpsk, 'k--', 'LineWidth',
2);
grid on;
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER Performance of QPSK (Gray vs Non-Gray)');
legend('Simulated Gray-coded', 'Simulated Non-Gray',
'Theoretical Gray-coded');
ylim([1e-4 1]);

%%
.....................................................8-
PSK.............................................%%

%.....................................Mapper.............
.....................%
% Manual 8PSK Gray labels
gray_labels = [
    0 0 0;  % 0
    0 0 1;  % 1
    0 1 1;  % 2
    0 1 0;  % 3
    1 1 0;  % 4
    1 1 1;  % 5
    1 0 1;  % 6
    1 0 0;  % 7
];

% Ideal 8PSK constellation points
angles = (0:7) * (2*pi/8);
constellation = exp(1j * angles);
% Group bits into symbols
eightpsk = groupBitsForMapping(data_bits, M_8PSK);
num_symbols = size(eightpsk,1);
% Map each group of 3 bits to a constellation point
```

```
psk8_symbols = zeros(num_symbols,1);
for k = 1:num_symbols
    idx = find(ismember(gray_labels, eightpsk(k,:),
'rows'));
    psk8_symbols(k) = constellation(idx);
end
%BER Simulation
BER_8psk = zeros(length(EbNo_dB), 1);
for k = 1:length(EbNo_dB)
    %............................. Add
AWGN.............................%
    rx_8psk = AWGN_Channel(psk8_symbols, EbNo_dB(k), Eb,
M_8PSK);
    % ..............Demapping: Find closest constellation
point..........%
    demapped_indices = zeros(length(rx_8psk),1);
    for i = 1:length(rx_8psk)
        [~, idx_min] = min(abs(rx_8psk(i) -
constellation)); % match with ideal
        demapped_indices(i) = idx_min - 1; % 0-indexed
    end
    % Convert symbol index to bits using Gray decoding
    demapped_bits = zeros(length(demapped_indices), 3);
    for i = 1:length(demapped_indices)
        demapped_bits(i,:) =
gray_labels(demapped_indices(i)+1,:);
    end

    demapped_bits = reshape(demapped_bits.', [], 1); %
column vector
    %......................... Calculate
BER.............................%
    BER_8psk(k) = BER_Calculation(demapped_bits, data_bits,
N);
end
%............................. Theoretical
BER............................%
theoretical_BER_8psk = (1/log2(8)) * erfc(sqrt(log2(8) *
10.^(EbNo_dB/10)) * sin(pi/8));
% Plot BER curves
figure;
semilogy(EbNo_dB, BER_8psk, 'r-', 'LineWidth', 2); hold on;
semilogy(EbNo_dB, theoretical_BER_8psk, 'k--', 'LineWidth',
2);
```

```
grid on;
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER Performance of 8-PSK');
legend('Simulated', 'Theoretical');
ylim([1e-4 1]);

%% ..............................................16-QAM
Modulation.....................................%%

%.............................Mapper.....................
.....%
Qam16 = groupBitsForMapping(data_bits, 16); % Group bits
into 4-bit symbols
% Convert data to Gray code
gray_bits = xor(Qam16, floor(Qam16 / 2));
% Convert Gray-coded bits to decimal values
gray_decimal = gray_bits(:,1)*8 + gray_bits(:,2)*4 +
gray_bits(:,3)*2 + gray_bits(:,4);
% Define 16-QAM Gray-coded mapping
Mapping = [ -3-3j, -3-1j, -3+3j, -3+1j, ...
            -1-3j, -1-1j, -1+3j, -1+1j, ...
             3-3j,  3-1j,  3+3j,  3+1j, ...
             1-3j,  1-1j,  1+3j,  1+1j];

% Map Gray decimal values to constellation points
qamSymbols = Mapping(gray_decimal + 1); % MATLAB indexing
+1
% BER Simulation
BER_qam = zeros(1, length(EbNo_dB));  % Preallocate
% 16-QAM Constellation
M16_QAM_constellation = Mapping.';
for i = 1:length(EbNo_dB)
    %..................... Transmit over AWGN
channel.....................%
    rx_qam = AWGN_Channel(qamSymbols, EbNo_dB(i), Eb,
M_QAM);
    %.........................
Demapping..............................%
    M16_QAM_demap = zeros(length(rx_qam), 1);
    for k = 1:length(rx_qam)
        [~, Min_index] = min(abs(rx_qam(k) -
M16_QAM_constellation));
        M16_QAM_demap(k) = Min_index - 1;
```

```
    end

    % Convert indices back to Gray-coded bits
    M16_QAM_demap = de2bi(M16_QAM_demap, 4, 'left-msb');
    M16_QAM_demap = reshape(M16_QAM_demap.', [], 1); %
column vector

    % ..................... BER
calculation.............................%
    BER_qam(i) = BER_Calculation(M16_QAM_demap, data_bits,
length(M16_QAM_demap));
end
% ................Theoretical BER for 16-
QAM.........................%
theoretical_BER_QAM = 3/8 *
erfc(sqrt(10.^(EbNo_dB/10)/(2.5)));

%  Plot results
figure;
semilogy(EbNo_dB, theoretical_BER_QAM, 'k--', 'LineWidth',
2);
hold on;
semilogy(EbNo_dB, BER_qam, 'r-', 'LineWidth', 2);
grid on;
legend('Theoretical BER', 'Simulated BER');
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER for 16-QAM Modulation');
ylim([1e-4 1]);

%% ..................................................BFSK
Modulation.....................................%%

%.....................................Mapper.............
.....................%
bfskSymbols = zeros(1 ,length(data_bits));

for k = 1:length(data_bits)
    if(data_bits(k) == 1)
        bfskSymbols(k) = sqrt(Eb) * complex(0,1);
    elseif(data_bits(k) == 0)
         bfskSymbols(k) = sqrt(Eb) * complex(1,0);
    end
end
```

```
rx_bfsk = zeros(length(bfskSymbols), 1);
BER_bfsk = zeros(length(EbNo_dB), 1);

for k = 1:length(EbNo_dB)
    %..................... Transmit over AWGN
channel.....................%
    rx_bfsk = AWGN_Channel(bfskSymbols, EbNo_dB(k), Eb, 0);
    %.........................
Demapping.............................%
    BFSK_demap = zeros(1, length(bfskSymbols));
    BFSK_constellation = [ complex(1,0);
                           complex(0,1);];

    for j = 1 : length(bfskSymbols)
        [~, Min_index] = min(abs(rx_bfsk(j) -
BFSK_constellation));
        BFSK_demap(j) = Min_index - 1;
    end

    % ..................... BER
calculation.............................%
    BER_bfsk(k) = BER_Calculation(BFSK_demap, data_bits,
length(BFSK_demap));
end

% ................Theoretical BER for
BFSK..........................%
theoretical_BER_bfsk = 0.5 * erfc( sqrt(0.5 *
10.^(EbNo_dB/10)) );

% Plot BER vs. Eb/No
figure;
semilogy(EbNo_dB, BER_bfsk, 'r-', 'LineWidth', 2);  hold
on;
semilogy(EbNo_dB, theoretical_BER_bfsk, 'k--', 'LineWidth',
2);
grid on;
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER Performance of Coherent BFSK');
legend('Simulated BER', 'Theoretical BER');
ylim([1e-4 1]);
```

```
%% 2.PSD
% Simulation parameters
bits = 200;                 % Number of bits per realization
realizations = 4000;        % Number of realizations for
ensemble averaging
Tb = 0.1;                   % Bit duration in seconds
Fs = 100;                   % Sampling frequency (samples
per second)
Ts = 1/Fs;                  % Sampling period
samples = Fs * Tb;          % Number of samples per bit
t = 0:Ts:(Tb-Ts);           % Time vector for one bit
duration

% Generate random binary data: dimensions (realizations x
(bits+1)*samples)
binary_data = randi([0, 1], realizations, bits+1);
binary_data = repelem(binary_data, 1, samples);  %repeating
the elements(bits)

% Preallocate arrays for modulated signals and ACF
BFSK_VAR1 = zeros(realizations, (bits+1)*samples); % Stores
modulated signal (before shifting)
BFSK_VAR2 = zeros(realizations, (bits*samples));   % Stores
signal after random shifting
BFSK_ACF = zeros(1, bits*samples);                 %
Autocorrelation function values

% Define BFSK modulated symbols (baseband representation)
% Reinitialization as we used the complex represantaion for
BER calculation
S1BB = sqrt(2*Eb/Tb);
S2BB = S1BB * (cos(2*pi*(1/Tb)*t) + 1i*sin(2*pi*(1/Tb)*t));


% Modulate data using BFSK scheme
indx = 1;
for i = 1:realizations
    for j = 1:(bits+1)*samples
        if binary_data(i,j) == 1
            BFSK_VAR1(i,j) = S1BB;
        else
            if indx > samples
                indx = 1;
```

```
            end
            BFSK_VAR1(i,j) = S2BB(indx);
            indx = indx + 1;
        end
    end
end


% Apply random circular shifts
for i = 1:realizations
    shift_i = randi([0, samples]);
    BFSK_VAR2(i, :) = BFSK_VAR1(i, shift_i+1 : shift_i +
bits*samples);
end

% Estimate Autocorrelation Function (ACF) by ensemble
average
for tau = (-bits*samples/2+1):bits*samples/2
     BFSK_ACF(tau + bits*samples/2)= sum(conj(BFSK_VAR2(:,
bits*samples/2)) .* BFSK_VAR2(:, bits*samples/2 + tau))/
realizations;
end


% Compute PSD by Fourier transform of the estimated ACF
fft_BFSK = abs(fft(BFSK_ACF));
L = length(BFSK_ACF);
f = (-L/2:L/2-1) * Fs/L *Tb;

% Compute theoretical PSD for BFSK (baseband)
f_theoretical = (-bits*samples/2:bits*samples/2-1) *
Fs/(bits*samples) ;
delta_1 = abs(f_theoretical - 0.5 / Tb) < Ts;
delta_2 = abs(f_theoretical + 0.5 / Tb) < Ts;
theoretical_psd = ((2/Tb) * (delta_1 + delta_2)) + ((8 *
cos(pi * Tb * f_theoretical).^2) ./ (pi^2 * (4 * Tb^2 *
f_theoretical.^2 - 1).^2));


% Plotting the graphs
figure;
plot(f, fftshift(fft_BFSK)/Fs,'b', 'linewidth', 1);
hold on;
plot(f_theoretical*Tb,theoretical_psd,'r', 'linewidth', 1);
```

```
title('PSD for BFSK scheme');
xlabel('1/Tb');
ylabel('Magnitude');
legend ('Simulated PSD', 'Theoretical PSD');
ylim ([0 2])

%% ..........................................Plot all
modulation types.................................%%
figure;
semilogy(EbNo_dB, BER_bpsk, 'r-', 'LineWidth', 2); hold on;
semilogy(EbNo_dB, BER_qpsk_gray, 'b-', 'LineWidth', 2);
semilogy(EbNo_dB, BER_qpsk_nongray, 'g-', 'LineWidth', 2);
semilogy(EbNo_dB, BER_8psk, 'm-', 'LineWidth', 2);
semilogy(EbNo_dB, BER_qam, 'k-', 'LineWidth', 2);
semilogy(EbNo_dB, BER_bfsk, 'y-', 'LineWidth', 2);
grid on;
legend('BPSK BER', 'Gray QPSK BER', 'Non Gray QPSK BER',
'8-PSK BER', '16-QAM BER', 'BFSK BER');
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER for all Modulation Types');
ylim([1e-4 1]);

%%
.............................................Functions.
.........................................%%
% function to arrange the data in form of 2*2 matrix
denpending of the mod
function bit_groups = groupBitsForMapping(data_bits, M)
    % Calculate bits per symbol
    bits_per_symbol = log2(M);

    % Ensure data_bits length is a multiple of
bits_per_symbol
    valid_length = floor(length(data_bits) /
bits_per_symbol) * bits_per_symbol;
    data_bits = data_bits(1:valid_length);

    % Reshape: each row is a symbol, each row has
bits_per_symbol bits
    bit_groups = reshape(data_bits, bits_per_symbol, []).';

    % Display summary
```

```
    fprintf('Data grouped into %d symbols, each with %d
bits.\n', size(bit_groups,1), bits_per_symbol);
end
```

```
% Adds AWGN noise to the transmitted signal
function rx_signal = AWGN_Channel(tx_signal, EbNo_dB, Eb,
M)

    if(M ~= 0)
    num_bits = log2(M); % bits per symbol
    EbNo_linear = 10.^(EbNo_dB/10); % Convert Eb/No from dB
to linear
    E_avg = mean(abs(tx_signal).^2);  % Signal power
    No = Eb ./ EbNo_linear; % N0 from Eb/N0 in linear scale
    noise_variance = No/2 * E_avg/num_bits; % Calculate
sigma squared
    else
        EbNo_linear = 10.^(EbNo_dB/10); % Convert Eb/No
from dB to linear
        No = Eb ./ EbNo_linear; % N0 from Eb/N0 in linear
scale
        noise_variance = No/2; % Calculate sigma squared
    end

    % Generate noise
    noise = sqrt(noise_variance) .* (randn(size(tx_signal))
+ 1j*randn(size(tx_signal)));

    % Add noise
    rx_signal = tx_signal + noise;
end

% Calculate the BER for the received signal
function BER = BER_Calculation(demapped_bits,
original_bits, N)

    num_errors = 0; % Number of errors (wrong bits) in the
matched filter output

    for k = 1:N
        % Loop through each bit and calculate number of
errors
        if(demapped_bits(k) ~= original_bits(k))
```

```
            num_errors = num_errors + 1;
        end
    end

    % Calculate BER
    BER = num_errors / N;
end
```